

# Ren'Py Deckbuilder

## Content Authoring Guide

How to build each level type from templates + the content library

Branch: **buildalpha** · Templates + library/manifest · Ren'Py still runs .rpy

This guide is for humans (and story bots). Copy a template, fill it with library assets, register ids, and jump in-game. JSON under **library/** is a catalog — Ren'Py does not load it at runtime.

# 1. Big picture

Three layers work together:

- **Templates** (game/scripts/templates/) — grammar for a new piece of content.
- **Library** (library/manifest.json) — stable ids of finished pieces a story can reuse.
- **Story spine** — thin .rpy that jumps/calls library ids (VN → RF chain → fight → return).

HUD → **Templates** opens `template_index` so you can try each kind live. Playtest art/story also lives on branch **vespera**; assembly work is on **buildalpha**.

---

## Golden rules

- Assets (images/audio) stay under `game/images/` and `game/audio/` — never inside `templates/`.
- Do **not** edit `follower_controller.rpy` or the battle loop to add content.
- Smart quotes in dialogue (What's, you're). Never use loop variable `_p` in `init python`.
- Never put background on a viewport. Do not add another `label start`.
- After adding art, restart Ren'Py (or clear cache) if a frame looks stuck.

## 2. Recommended image sizes

Target these when exporting. Battle tags use ysize=700 in-game (aspect preserved).

| Asset                        | Size / format             | Notes                                                                   |
|------------------------------|---------------------------|-------------------------------------------------------------------------|
| RF map background            | 1920 × 1080, JPG/PNG      | Opaque OK. One image per room.                                          |
| RF walker frames             | 960 × 960 PNG (RGBA)      | Cutout on transparent. Display zoom ~0.22 (crouch 0.16).                |
| RF city / map pin            | ~64 × 64 PNG              | Token on city map / interact pin.                                       |
| Battle sprites               | 700 px tall (or 1400 @2×) | Portrait crop; width ~450–550 typical. Square 2048 wastes screen width. |
| Dialogue bust                | ~600 × 700 PNG            | Chest-up, mask on. Becomes side image emotions.                         |
| Dialogue / capture full-body | ~700 × 1000 PNG           | Standing / capture stills.                                              |
| Enemy battle art             | Match player height band  | Needs enemies/{name}.png + {name} hover.png.                            |
| Story CG / fullscreen show   | 1920 × 1080               | Opaque fine for BGs; PNG if you need alpha.                             |

**Battle tip:** Export at 700 px height so in-game scale is 1:1. Replacing files under game/images/vespera/battle/ is enough — no code change unless height target changes.

**RF tip:** Keep cutout feet near the bottom of the frame. Engine plants the sprite bottom on the path node.

### 3. Standard Ren'Py dialogue (VN)

**Template:** game/scripts/templates/vn\_scene.rpy → jump template\_vn

#### Steps

1. Copy the template file to e.g. game/scripts/story/my\_scene.rpy.
2. Rename the Character define and the label (avoid clashing with template\_vn).
3. Set scene image (scene bg ...), show sprites/busts, write dialogue + menu.
4. Jump/call targets: back to an RF room label, another VN beat, or start\_fight.
5. Clear RF/battle UI at the top (copy the hide-screen block from the template).

#### Wire-up

```
label my_scene:
    hide screen rf_map
    # ... hide battle screens ...
    scene bg rooftop night with fade
    $ show_hud()
    vespera "Line here."
    menu:
        "Head back":
            jump rooftop_a_1
```

Add the beat to library/manifest.json → **vn\_beats** when it's reusable.

## 4. RF — single image

**Template:** game/scripts/templates/rf\_single.rpy → template\_rf\_single

One background + one node map + optional interact points. Live example: rooftop\_a\_1.

### Steps

1. Drop a 1920×1080 map under game/images/maps/...
2. Build the route in the RF editor (Save Route). Paste into a default `my_route = [...]` list — don't hand-type huge arrays.
3. Author interact points (name, point, label, active, detected, once + optional fields).
4. Register the room:

```
init 1 python:
    rf_register_room(
        "my_room",
        bg="maps/my/map.jpg",
        route="my_route",
        start=(x, y),          # must sit on a route node
        points="my_points",    # or None
    )
```

5. Entry label:

```
label my_room:
    $ current_rf_level = "my_room"
    jump rf_play
```

6. Add library/rf\_rooms/my\_room.json + a manifest entry.

### Interact point cheat-sheet

- label: "rf\_npc" — side image + lines
- rf\_pickup / rf\_dialogue / rf\_show — built-ins in rf\_points.rpy
- Custom label — your VN or `$ start_fight("id")`, then jump back to the room

## 5. RF — chain of images

**Template:** game/scripts/templates/rf\_chain.rpy → template\_rf\_chain

A level is several rooms. You **jump once into the chain**; room swaps stay inside the chain file / library API. VN and battles are temporary side trips that return to a room enter label.

### Steps

1. Author each room as in §4 (bg + route + points + rf\_register\_room).
2. Register the chain:

```
init 1 python:
    rf_register_chain(
        "my_level",
        entry="room_a",
        rooms=["room_a", "room_b", "room_c"],
        exits={"room_a", "next": "room_b", ("room_b", "prev"): "room_a"},
    )
```

3. Chain entry label (players / story bot jump here):

```
label my_level:
    $ current_rf_chain = "my_level"
    $ current_rf_level = "room_a"
    jump rf_play
# or: $ rf_enter_chain("my_level")
```

4. Exit interact on a room calls:

```
label room_a_to_b:
    $ rf_chain_exit("room_b")
```

Live example: jump rooftop\_a (rooms 1→2→3). Catalog: library/rf\_chains/rooftop\_a.json.

## 6. Single battle

**Template:** game/scripts/templates/battle\_single.rpy → template\_battle\_single

One-off named fight (battle\_mode "instance"). Does **not** advance the arena ladder.

### Steps

1. Copy the FIGHTS dict into game/scripts/data/fights.rpy (or keep in your story file with a guard).
2. Set id, scene, on\_win, on\_lose, enemies[].
3. Enemy images under game/images/enemies/ — need {name}.png and {name}\_hover.png.
4. Start from RF/VN:

```
$ start_fight("my_bout")  
# on_win / on_lose labels fire after battle
```

Action keys: say, attack, heal, anim (kick/punch/slash/cast/raise\_hand), status + duration/stacks, stun + duration.

List the fight under library/fights/ + manifest when stable.

## 7. Battle arena

**Template:** game/scripts/templates/battle\_arena.rpy → template\_battle\_arena

Ladder in game/scripts/data/levels.json. Wins call levels.next().

### Steps

1. Edit levels.json keys "0", "1", ...
2. Each entry: scene + enemies (authored actions **or** attack\_min/max heal\_min/max).
3. Jump template\_battle\_arena (sets arena mode, level 0, jumps battle).

Do not mix instance fights with ladder progression — use §6 for story bosses.

## 8. Side trips: VN / battle from RF

From an interact point label:

- **VN:** `jump my_scene` (or `call` if it should return automatically). Hide `rf_map`, play, then jump the same room again.
- **Battle:** `$ start_fight("id")` with `on_win/on_lose` set to room or VN labels.

The chain file stays home base; side content is temporary.

## 9. Promoting into the library

When a piece is reusable:

1. Ensure it has a stable id and enter label / FIGHTS key.
2. Add `library/.../id.json` describing paths and how to start it.
3. Append the id to `library/manifest.json`.
4. Story bots must only use ids listed in the manifest.

Runtime API (from `game/RF/library_loader.rpy`): `rf_register_room`, `rf_register_chain`, `rf_load_room`, `rf_enter_chain`, `rf_chain_exit`, `rf_current_bg`.

## 10. Quick checklist per new piece

- Art at recommended size, correct folder
- Template copied / fields filled
- Registered (room / chain / FIGHTS / Character)
- Enter label works from HUD or story jump
- Manifest + library JSON updated
- `on_win` / `on_lose` / menu jumps land somewhere real

---

Further reading: [TEMPLATES.md](#), [LIBRARY.md](#), [library/manifest.json](#).  
Generated for AM Photography · Ren'Py Deckbuilder · buildalpha